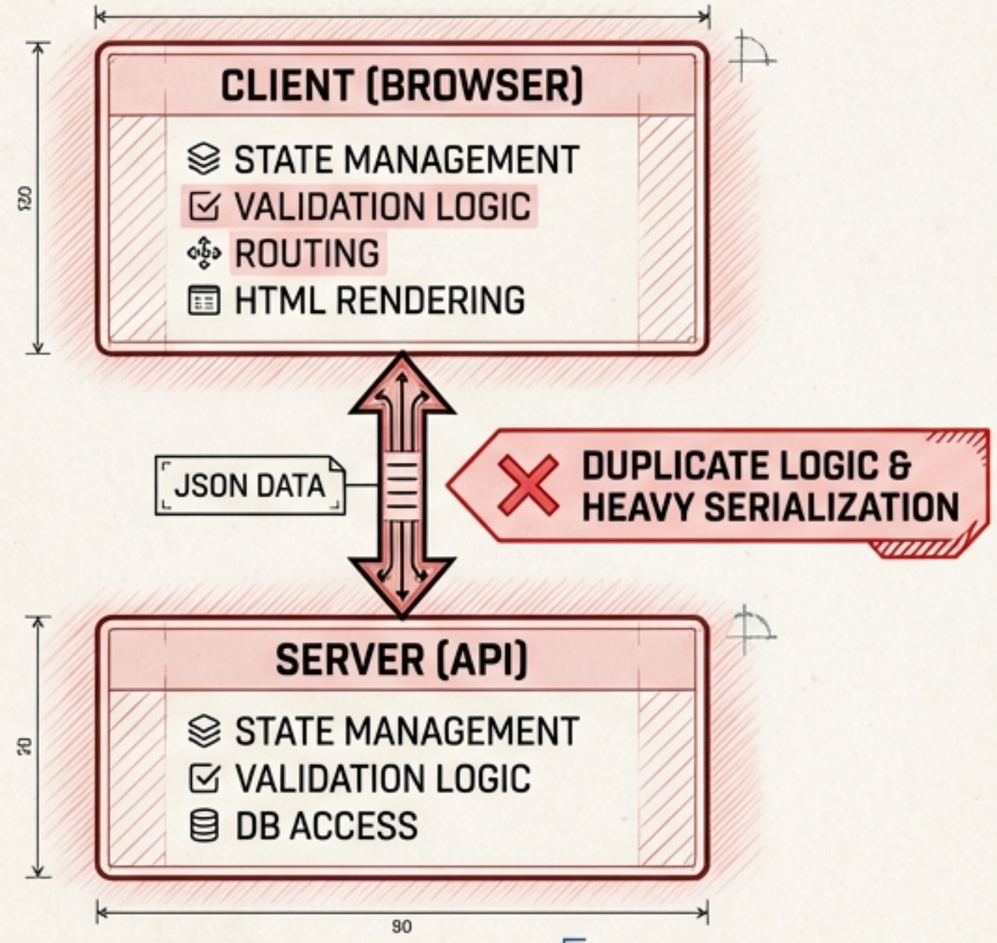
Replacing React with Go & HTMX

Building Modern SPAs with Hexagonal Architecture

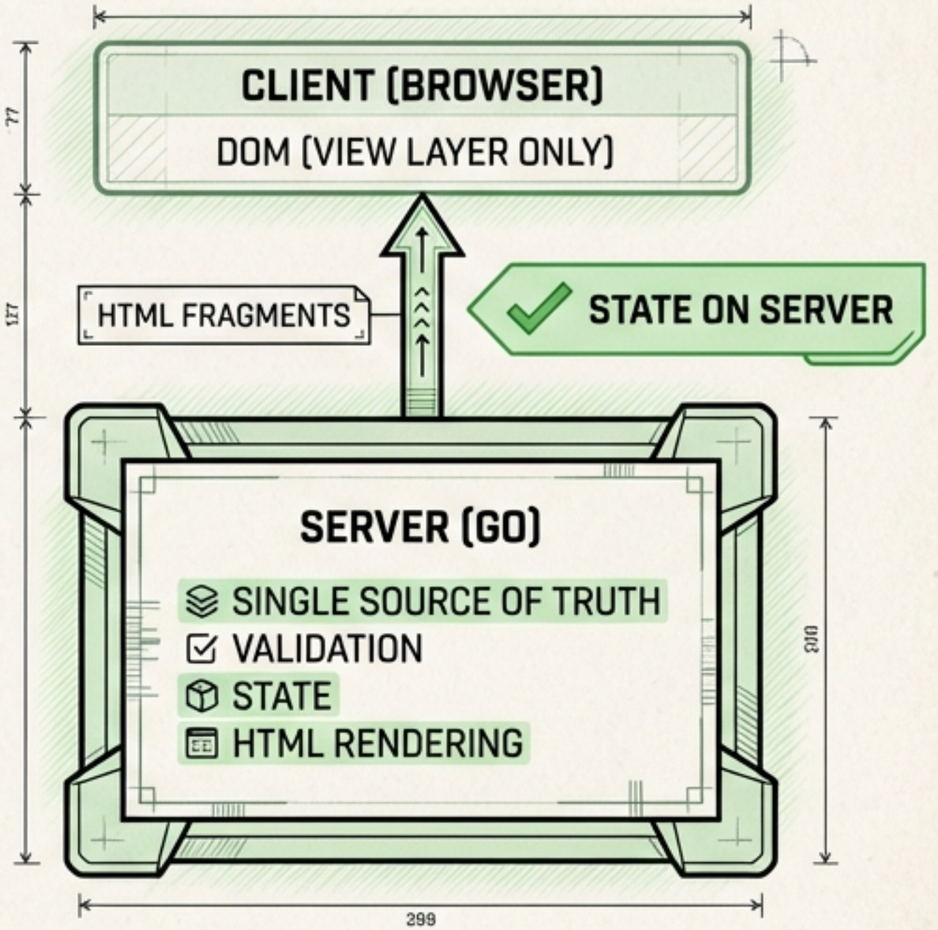
CASE STUDY: GOKANBAN | ZERO CUSTOM JAVASCRIPT | 100% SERVER-SIDE STATE

THE COMPLEXITY TRAP OF MODERN SPAs

CURRENT STANDARD: **REACT/VUE**



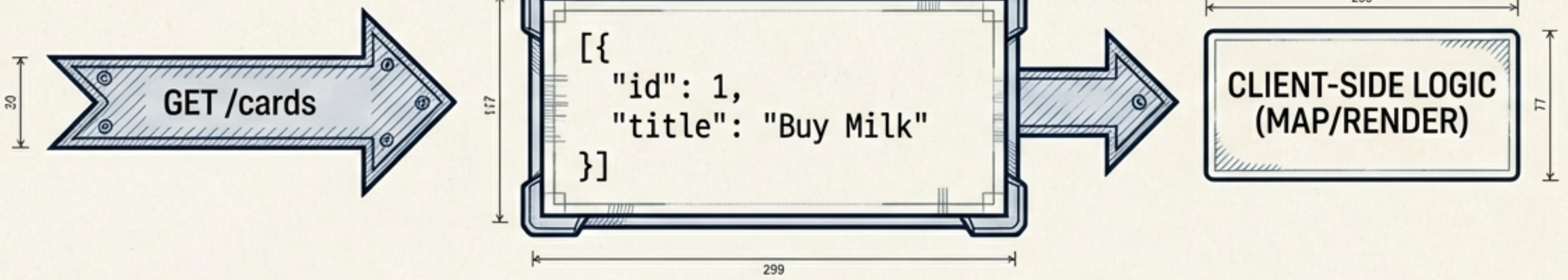
PROPOSED: **HYPERMEDIA**



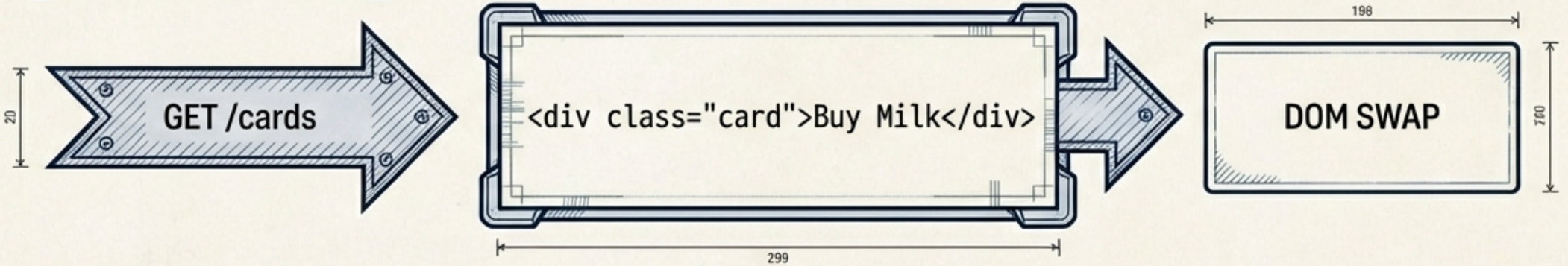
STOP SENDING JSON JUST TO RE-RENDER IT ON THE CLIENT.
MOVE THE STATE BACK TO WHERE IT BELONGS.

THE HTMX MINDSET: HYPERMEDIA AS ENGINE OF STATE

TRADITIONAL JSON API

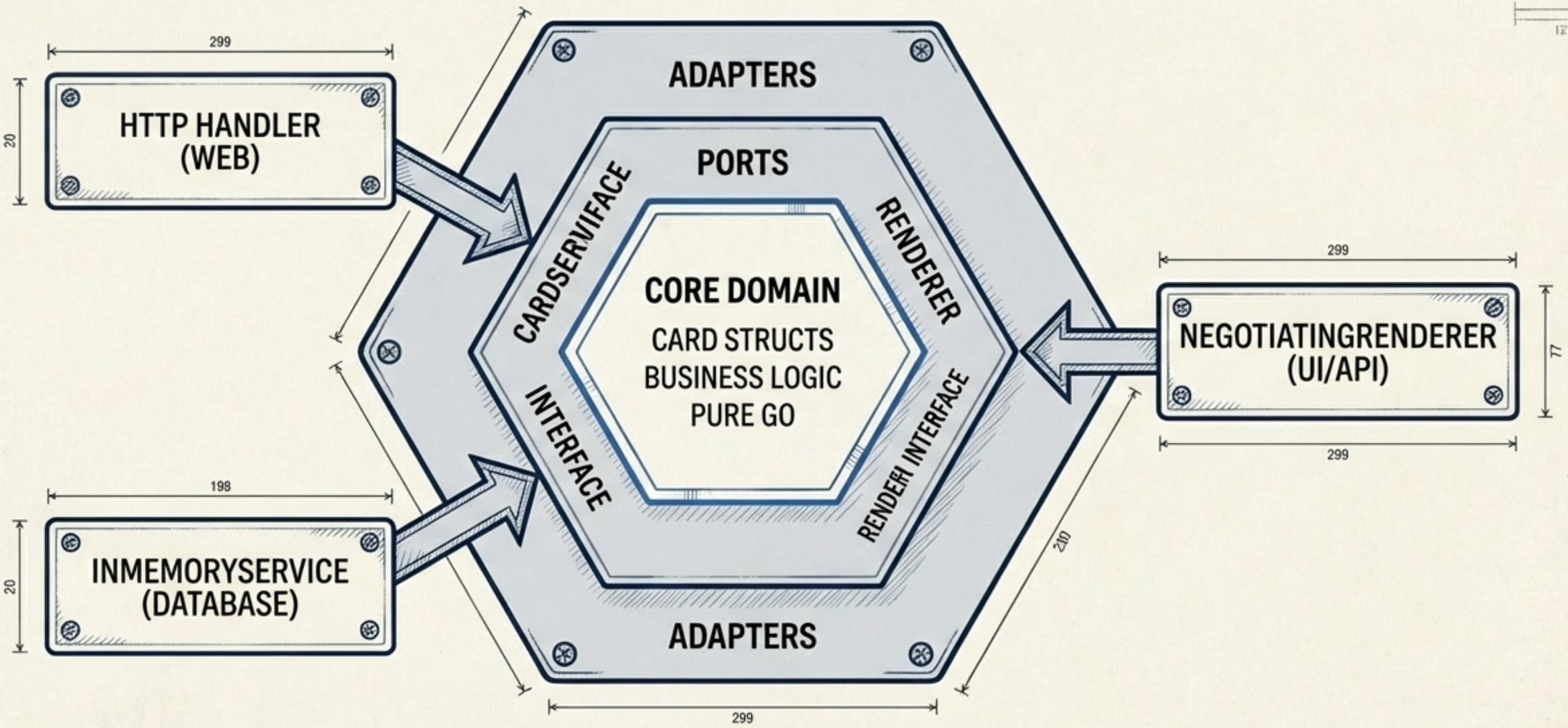


HTMX APPROACH



"THE DOM IS MERELY A REFLECTION OF SERVER STATE.
WE DON'T FETCH DATA; WE FETCH THE UI."

HEXAGONAL ARCHITECTURE: PORTS & ADAPTERS



DECOUPLING THE CORE LOGIC FROM THE RENDERING FORMAT ENSURES THE BACKEND REMAINS CLEAN.

THE NEGOTIATING RENDERER

```
func (n *NegotiatingRenderer) Render(
      w http.ResponseWriter, r *http.Request,
      ctx RenderContext) {

    // Check if client wants API data
    if r.Header.Get("Accept") == "application/json" {
        json.NewEncoder(w).Encode(ctx.Data) // API Mode
        return
    }

    // Otherwise, serve HTMX/HTML
    w.Header().Set("Content-Type", "text/html")
    n.tmpl.ExecuteTemplate(w, ctx.TemplateName, ctx.Data)
}
```

Traffic Control. This line decides if the app behaves as an API or a Website.

Adapter A: Supports Mobile Apps & CLI tools.

Adapter B: Supports Browsers & HTMX.

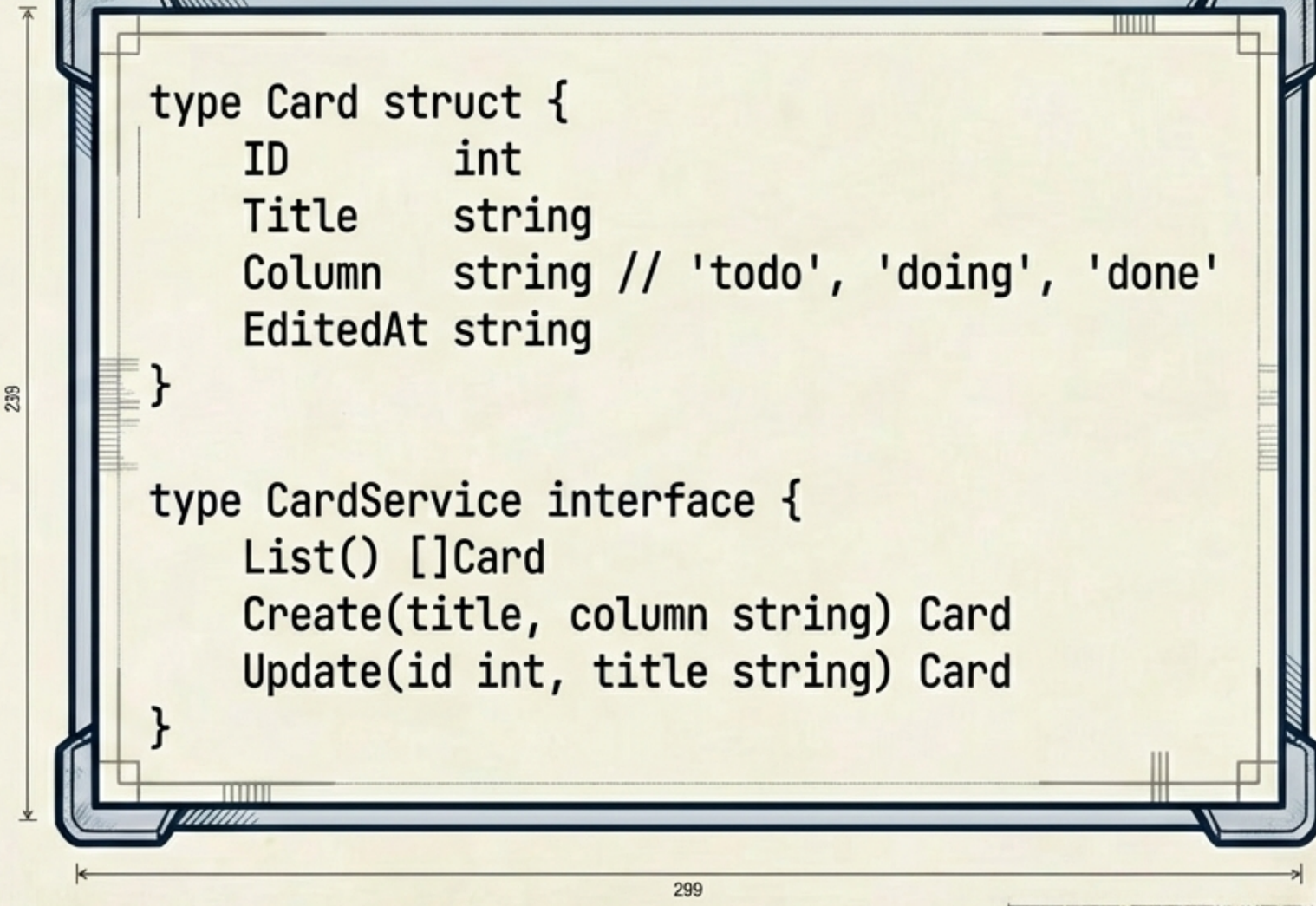
"One backend, two outputs. No code duplication."

THE DOMAIN LAYER: PURE GO

No HTTP dependencies. No JSON marshalling logic cluttering the core. Just data structures and behavior.

```
type Card struct {
    ID       int
    Title    string
    Column   string // 'todo', 'doing', 'done'
    EditedAt string
}

type CardService interface {
    List() []Card
    Create(title, column string) Card
    Update(id int, title string) Card
}
```

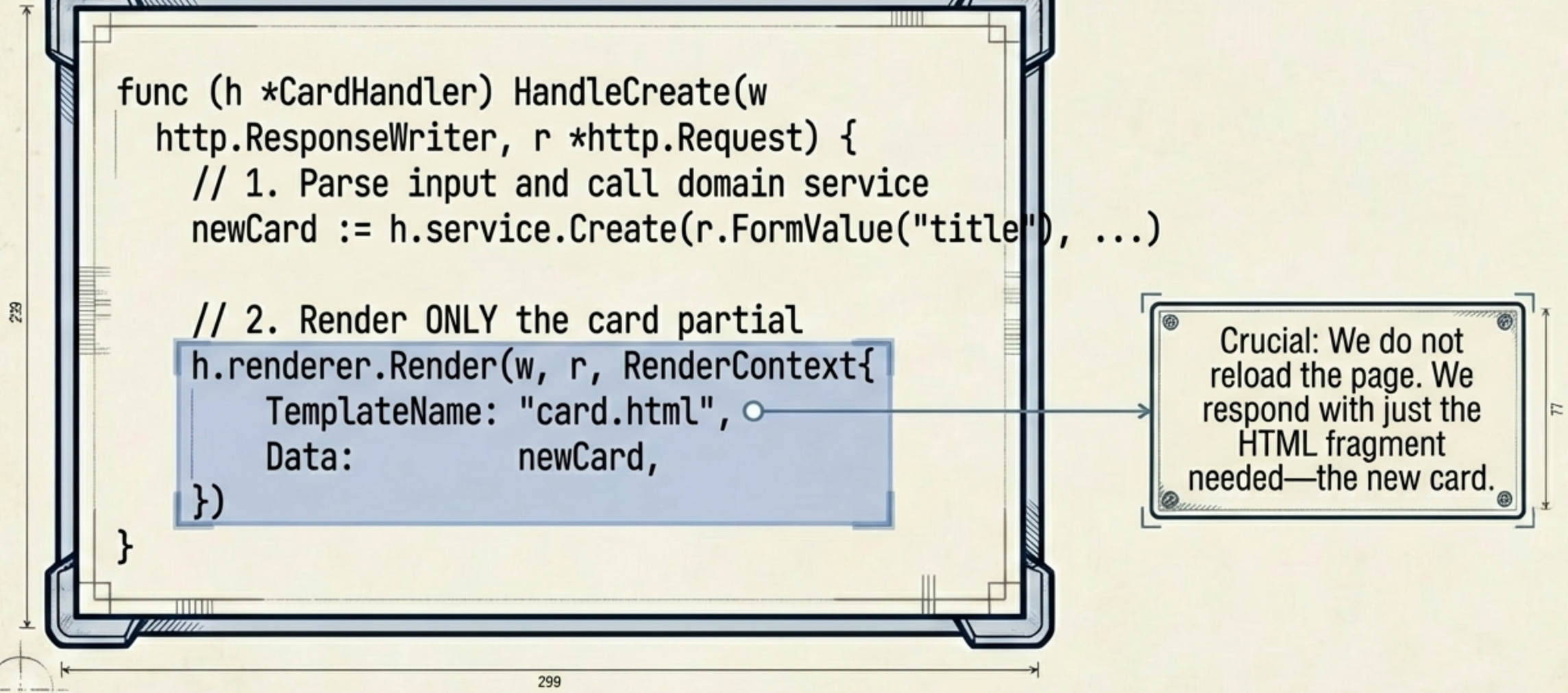


WIRING THE HANDLERS

```
func (h *CardHandler) HandleCreate(w
  http.ResponseWriter, r *http.Request) {
    // 1. Parse input and call domain service
    newCard := h.service.Create(r.FormValue("title"), ...)

    // 2. Render ONLY the card partial
    h.renderer.Render(w, r, RenderContext{
        TemplateName: "card.html",
        Data:         newCard,
    })
}
```

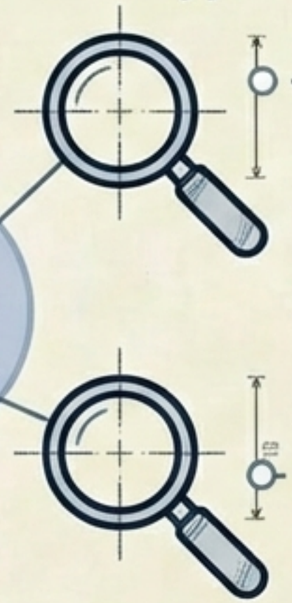
Crucial: We do not reload the page. We respond with just the HTML fragment needed—the new card.



THE FRONTEND: LAYOUT & THE TARGET

```
index.html
<!-- The Container -->
<div id="todo-list"> {{ range .Cards }} . . .
</div>

<!-- The Trigger -->
<form hx-post="/cards"
      hx-target="#todo-list"
      hx-swap="beforeend">
    <input name="title">
    <button>Add</button>
</form>
```



hx-target: Tells HTMX where to put the server's response (the container ID).

hx-swap: Tells HTMX to append the new card at the end, rather than replacing the entire list.

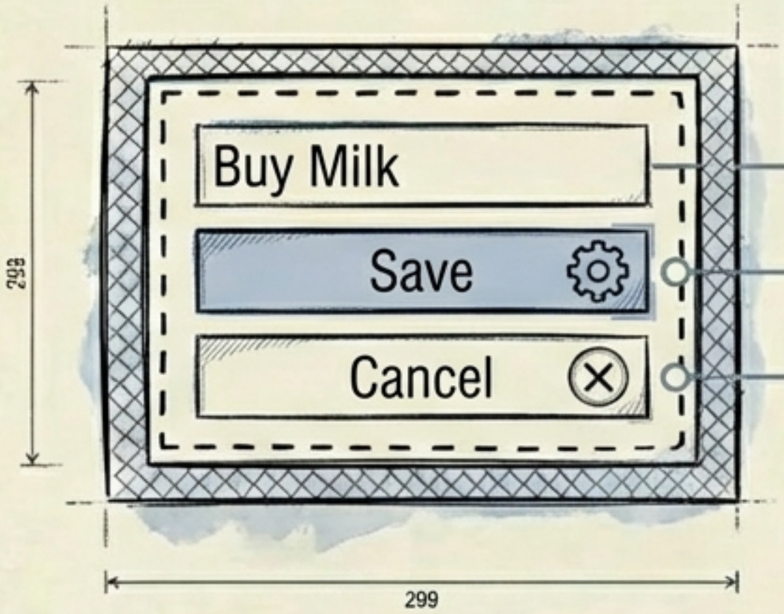
299

VIEW STATE: THE CARD COMPONENT


```
<div id="card-{{.ID}}"
   hx-get="/cards/{{.ID}}/edit"
   hx-trigger="click"
   hx-target="this"
   hx-swap="outerHTML">
   {{.Title}}
</div>
```

1. **Trigger**: The card itself is the button (**hx-trigger="click"**).
2. **Action**: It asks the server for its own “Edit Mode” (**hx-get**).
3. **Swap**: It replaces itself entirely (**hx-swap="outerHTML"**).

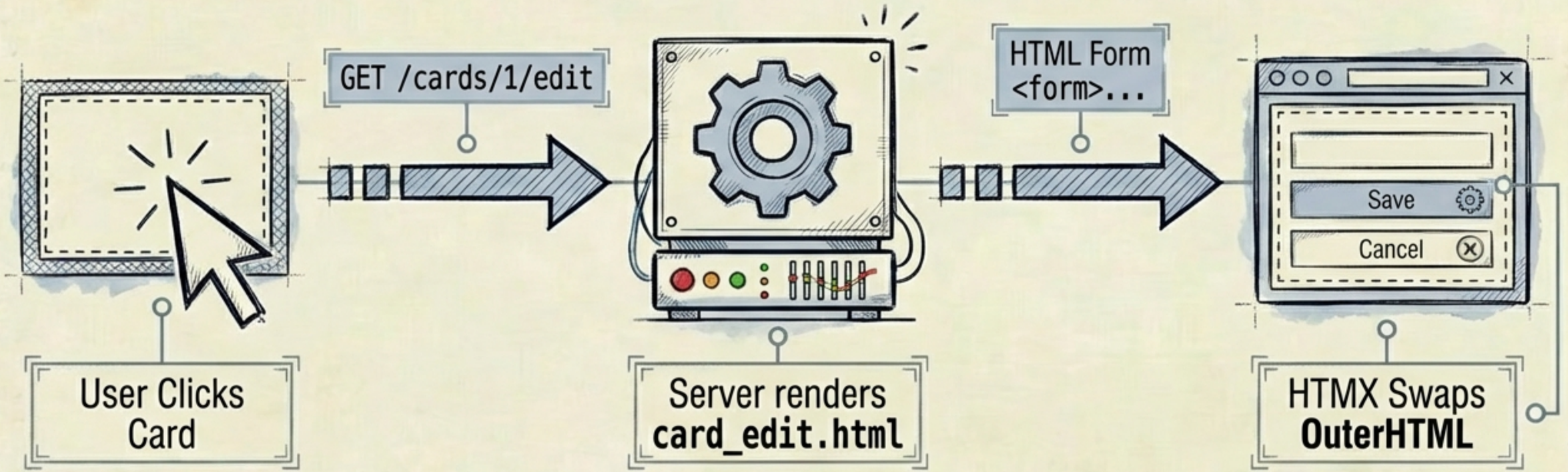
EDIT STATE: THE FORM COMPONENT




```
<form hx-put="/cards/{{.ID}}"
    hx-target="this" hx-swap="outerHTML">
    <input type="text" name="title"
     value="{{.Title}}">
    <button type="submit">Save</button>
    <button hx-get="/cards/{{.ID}}">Cancel
</form>
```

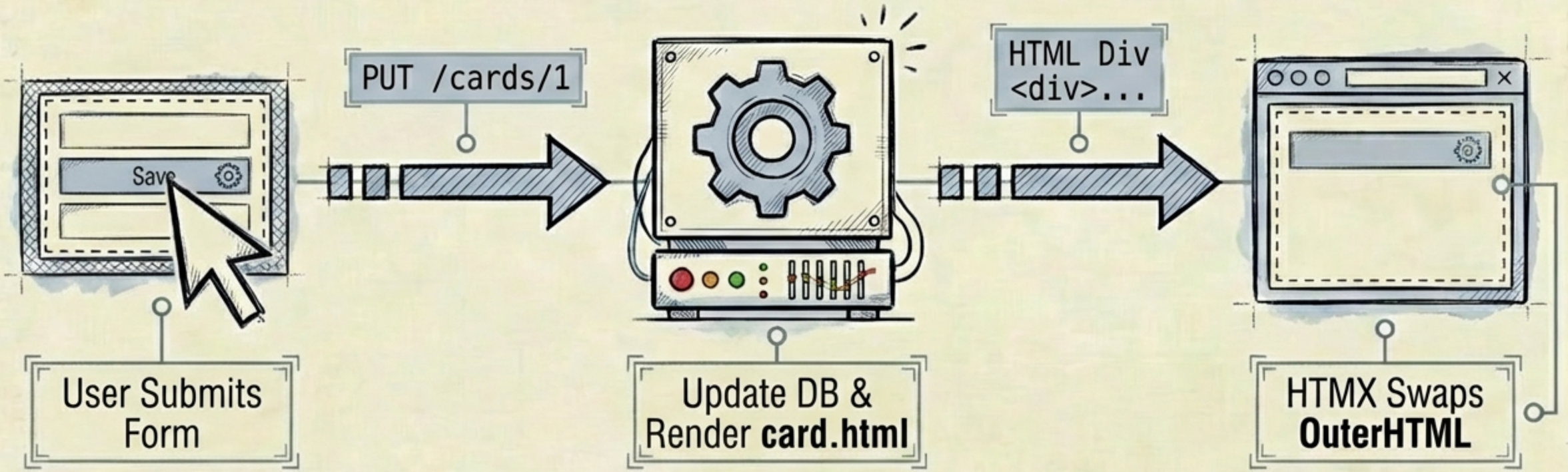
When submitted, this form sends the update to the server. The server responds with the *original* View State (**card.html**), swapping the UI back to non-editable mode.

VISUALIZING THE 'EDIT' TRANSITION



Inline editing achieved without `useState` or client-side routing.

VISUALIZING THE ‘SAVE’ TRANSITION

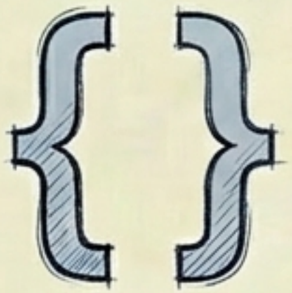


The server dictates the new state of the UI.

The GoKanban Stack

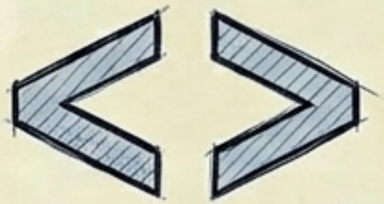
Language: Go

Strong typing, concurrency, native standard library.



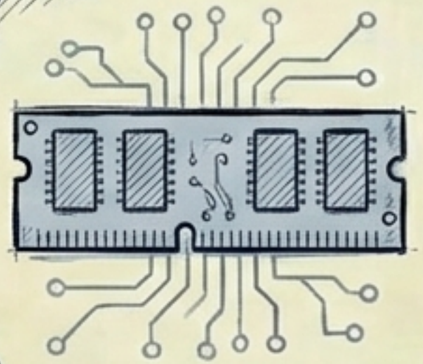
Templating: html/template

Native Go library. Fast, secure, zero dependencies.



Interactivity: HTMX

Hypermedia attributes. Access AJAX, CSS Transitions, and WebSockets directly in HTML.



Data: In-Memory

Thread-safe struct storage with `sync.Mutex`. `con.tiʁn`. No external DB required for demo.

Total Deployment Size: Single compiled binary

WHY THIS MATTERS

01

Unified State

Validation and logic live in one place (Go), not duplicate across Client and Server.

02

Hexagonal Flexibility

The same backend serves HTML to browsers and JSON to mobile apps via content negotiation.

03

Zero JavaScript Complexity

Dynamic, inline editing and SPA-feel without a build pipeline, `node_modules`, or client-side routing.

STOP BUILDING TWO APPLICATIONS.

You don’t always need a separate frontend and backend codebase. When the server drives the state, the complexity remains where you have the best tools to manage it.

RESOURCES

- Code: github.com/user/gokanban
- HTMX: htmx.org
- Go: go.dev